

PHP, módulo 4

Programación orientada a objetos

Programación orientada a objetos

- Objetos y clases. Repaso
- Cómo crear clases, atributos y operaciones en PHP
- Qué es el constructor de una clase
- Cómo usar los métodos de una clase
- Qué es la Herencia en PHP y como implementarla
- Librerías gráficas y multimedia en PHP (GD, JpGraph, Flash, FDF).
- Ejercicios: Realizamos 11 ejercicios:
- Ej.1 Creación de clases, atributos y métodos
- Ej.2 Creación de clases, uso del constructor
- Ej.3 Creación de clases, invocación o llamado a varios métodos
- Ej.4 Creación de clases, herencia
- Ej.5 Hacer conexiones con programación orientada a objeto
- Ej.6 Combos dinámicos desplegados enlazados
- Ej.7 Gráfico de línea
- Ej.8 Gráfico de torta
- Ej.9 Gráfico de barra
- Ej.10 Imprimir en PDF
- Ej.11 Foro por temas

Objetos y clases. Repaso.

¿Que es la programación orientada a objetos?



La programación Orientada a objetos (POO) es una forma especial de programar, más cercana a como expresaríamos las cosas en la vida real que otros tipos de programación, hay que destacar que en la POO todo se define como un objeto.

Objetos y clases. Repaso.

Pero.. y ¿Que es un Objeto?



Un objeto en la programación se define como un clase que contiene atributos y métodos, esta tiene la posibilidad de heredar datos de otras y también poder definir algunos métodos especiales.

Objetos y clases. Repaso.

Ejemplo Básico

Te presento a Pedro..



Nombre
Edad
Sexo

Objetos y clases. Repaso.

¿Cual es la diferencia en PHP?



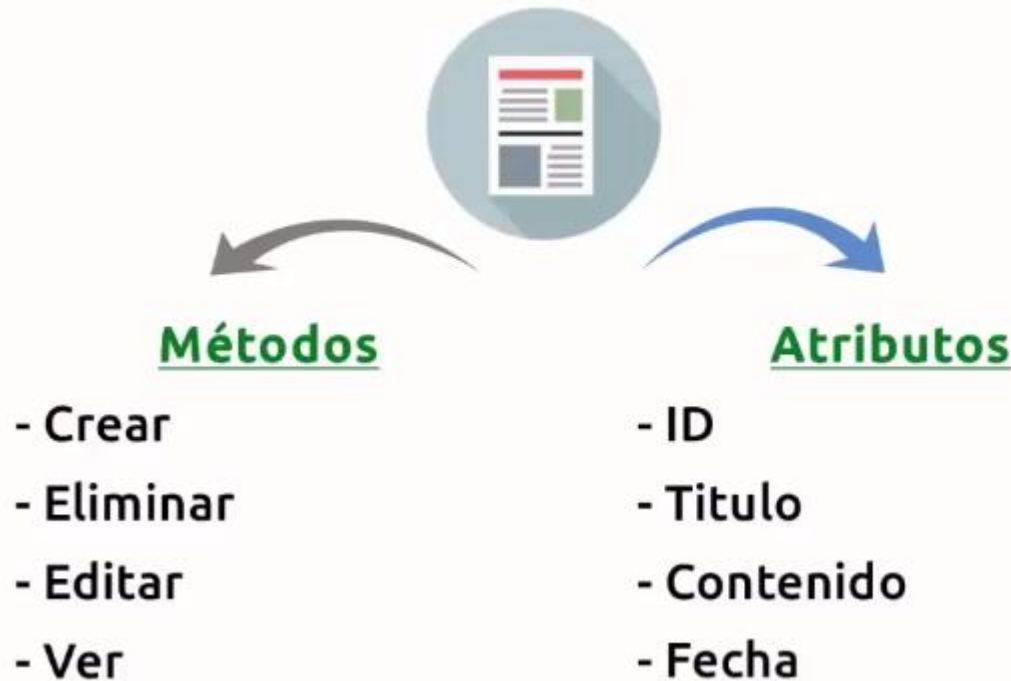
Programación Lineal: Es cuando desarrollamos todo el código disponiendo instrucciones PHP alternando con el HTML de la página.

Programación Estructurada: Es cuando planteamos funciones que agrupan actividades a desarrollar y luego dentro de la página llamamos a dichas funciones que pueden estar dentro del mismo archivo o en una librería separada.

Programación Orientada a Objetos: Es cuando planteamos clases y definimos objetos de las mismas.

Objetos y clases. Repaso.

Ejemplo en la programación



Cómo crear clases, atributos y operaciones en PHP

```
<?php
```

```
// Definición de la clase
```

```
class Persona
```

```
{
```

```
    // Atributos
```

```
    public $nombre = "Pedro";
```

```
    // Métodos
```

```
    public function hablar($mensaje)
```

```
    {
```

```
        echo $mensaje;
```

```
    }
```

```
}
```


Cómo crear clases, atributos y operaciones en PHP

// Creación del objeto

```
$persona = new Persona();
```

// Acceso al atributo

```
//echo $persona->nombre;
```

// Acceso al método

```
$persona->hablar("Saludos desde PHP módulo 4");
```

// GUARDAR COMO index.php, EN c:\xampp\htdocs\micarpeta

?>

Cómo crear clases, atributos y métodos en PHP

Atributos & Métodos

Atributos

- Simples Variables o Arrays
- Definición del tipo del atributo:

(Public, Private, Protected)

Métodos

- Accede a atributos con `$this->Nombre`
- Podemos acceder a otros métodos con `$this` o `self::`

Cómo crear clases, atributos y métodos en PHP

Ejercicio a realizar



- Tenemos una escuela donde registraremos nombre y apellido de cada persona.



- Usaremos 2 atributos que son su nombre y apellido y 2 métodos que son guardar y mostrar.

Cómo crear clases, atributos y métodos en PHP

```
<?php
class Persona
{
    //Atributos
    public $nombre = array();
    public $apellido = array();

    //Métodos
    public function guardar($nombre, $apellido)
    {
        $this->nombre[] = $nombre;
        $this->apellido[] = $apellido;
    }
}
```

Cómo crear clases, atributos y métodos en PHP

```
public function mostrar()  
{  
    for($i = 0; $i < count($this->nombre); $i++)  
    {  
        self::formato($this->nombre[$i], $this->apellido[$i]);  
        // Persona::formato($this->nombre[$i], $this->apellido[$i]);  
        // $this->formato($this->nombre[$i], $this->apellido[$i]);  
    }  
}  
  
public function formato($nombre, $apellido)  
{  
    echo "Nombre: " . $nombre . " | Apellido: " . $apellido . "<br>";  
}  
}
```

Cómo crear clases, atributos y métodos en PHP

```
$persona = new Persona();  
    $persona->guardar("Juan","Gamez");  
    $persona->guardar("Victor","Rodriguez");  
    $persona->guardar("Henry","Duque");  
    $persona->mostrar();
```

// GUARDAR COMO index.php, EN c:\xampp\htdocs\micarpeta

?>

Métodos: `__construct` y `__destruct`

¿Que son este par de métodos?



El método constructor y el destructor son un par de **métodos mágicos** de PHP muy usados cuando desarrollamos con la POO.

`__construct`

- Al instanciar la clase el método se ejecuta al comienzo de manera automática

`__destruct`

- Este método también se ejecuta de manera automática pero al final de la clase.

Métodos: __construct y __destruct

Ejemplo practico

- Imaginemos que tenemos que construir un hogar de arriba hacia abajo, necesitamos un techo, el contenido y por supuesto la base de nuestra casa.

- **Constructor**



- **Métodos**



- **Destructor**



Métodos: __construct y
__destruct

Ejercicio propuesto



- Vamos a realizar un aplicación tipo lotería donde le indicaremos un numero aleatorio y la cantidad de intentos, esta debe mostrar los intentos y al final debe mostrar un mensaje si gano o no el concursante.

Debemos usar los métodos __construct & __destruct

Métodos: __construct y __destruct

```
<?php
class Loteria {
    // Atributos
    public $numero;
    public $intentos;
    public $resultado = false;

    // Métodos
    public function __construct($numero, $intentos){
        $this->numero = $numero;
        $this->intentos = $intentos;
    }
}
```

Métodos: __construct y __destruct

```
public function sortear(){
    $minimo = $this->numero / 2;
    $maximo = $this->numero * 2;

    for ($i = 0; $i < $this->intentos; $i++){
        $int = rand($minimo,$maximo);
        self::intentos($int);
    }
}

public function intentos($int){
    if($int == $this->numero){
        echo "<b>" . $int . " == " . $this->numero."</b><br>";
        $this->resultado = true;
    }else{
        echo $int . " != " . $this->numero."<br>";
    }
}
```

Metodos: __construct y __destruct

```
public function __destruct(){
    if ($this->resultado){
        echo "Felicidades, has ganado en ". $this->intentos."
intentos";
    }else{
        echo "Que lastima, has perdido en ". $this-
>intentos." intentos";
    }
}

}

$loteria = new Loteria(10,5);
$loteria->sortear();
// GUARDAR COMO index.php, EN c:\xampp\htdocs\micarpeta
?>
```

Modificadores de acceso

¿Que son los modificadores de acceso?



Son simples propiedades que podemos añadirles a nuestros métodos y atributos de nuestra clase, estas nos ayudaran con los accesos y la extracción de métodos o atributos.

Modificadores de acceso

Tipos



Public

Esta propiedad es muy simple, se puede realizar de todo con el atributo o el método.



Private

Un método privado (private) solo puede ser llamado desde otro método de la clase. No podemos llamar a un método privados desde donde definimos un objeto.



Protected

Un atributo o método protected puede ser accedido por la clase, por todas sus subclases pero no por los objetos que definimos de dichas clases.

Modificadores de acceso

Ejemplo Básico



Imaginemos una cuenta de Facebook, hay datos que pueden ser mostrados pero otros no, ejemplo la contraseña de nuestra cuenta.



El nombre, apellido y su edad son visibles para todos.



La contraseña no, además solo puede ser editada por el mismo usuario.

Modificadores de acceso

```
<?php
class Facebook {
    //Atributos
        public $nombre;
        public $edad;
        private $pass;

    //Métodos
        public function __construct ($nombre, $edad, $pass){
            $this->nombre = $nombre;
            $this->edad = $edad;
            $this->pass = $pass;
        }
}
```


Modificadores de acceso

```
public function verInformacion (){
    echo "Nombre:". $this->nombre. "<br>";
    echo "Edad:". $this->edad. "<br>";
    //self::cambiarPass("54321");
    $this->cambiarPass("54321");
    echo "Pass:". $this->pass. "<br>";
}
private function cambiarPass($pass){
    $this->pass = $pass;
}
}
```

Modificadores de acceso

```
$facebook = new Facebook("Jhon Smith",32,"1234");  
//$facebook->verInformacion();  
/* Aqui puedo ver una variable publica*/  
/**echo $facebook->nombre;  
/* pero aqui no puedo ver una variable privada o  
private y solo puede verse desde otro metodo  
echo $facebook->pass;*/  
$facebook->verInformacion();
```

?>

Herencia

¿De que se trata la herencia?



La herencia significa que se pueden crear nuevas clases partiendo de clases existentes, que tendrá todas los atributos y los métodos de su **superclase** o **clase padre** y además se le podrán añadir otros atributos y métodos propios.

En PHP, a diferencia de otros lenguajes orientados a objetos (C++), una clase sólo puede derivar de una única clase, es decir,

PHP no permite herencia múltiple.

Herencia

Ejemplo básico



Vehículo



Autobús



Motocicleta

Herencia

```
<?php
class Vehiculo {
    //Atributos
        public $motor=false;
        public $marca;
        public $color;
    //Metodos
        public function estado() {
            if($this->motor == true)
                echo "El motor esta encendido<br>";
            else
                echo "El motor esta apagado<br>";
        }
}
```

Herencia

```
public function encender(){  
    if ($this->motor == true){  
        echo "Cuidado, el motor ya esta  
encendido<br>";  
    }else{  
        echo "El motor ahora esta  
encendido<br>";  
        $this->motor = true;  
    }  
}
```

Herencia

```
class Moto extends Vehiculo
{
    public function estadoMoto()
    {
        $this->estado();
    }
}
```

```
class CuatriMoto extends Moto
{
}
}
```

Herencia

```
/* Primer Bloque de Pruebas Class Vehiculo
```

```
$vehiculo = new Vehiculo();
```

```
$vehiculo->estado();
```

```
$vehiculo->encender();
```

```
$vehiculo->estado();
```

```
*/
```

```
/* Segundo Bloque de Pruebas Class Moto */
```

```
$moto = new Moto();
```

```
$moto->estado();
```

```
/*
```

```
$moto->encender();
```

```
$moto->estado();
```

```
*/
```

```
/*Tercer Bloque de Pruebas Class Cuatrimoto
```

```
$moto = new CuatriMoto();
```

```
$moto->estado();
```

```
*/
```

```
// GUARDAR COMO index.php, EN c:\xampp\htdocs\micarpeta
```

```
?>
```


Métodos y atributos estáticos

¿Que es esto?



Un método estático pertenece a la clase pero no puede acceder a los atributos de una instancia. La característica fundamental es que un método estático se puede llamar sin tener que crear un objeto de dicha clase.

Una propiedad declarada como static no puede ser accedida con un objeto de clase instanciado (**aunque un método estático sí lo puede hacer**).

Métodos y atributos estáticos

Lo que debemos recordar...

- Un método estático no puede acceder a los atributos de la clase.
- Indicamos primero el nombre de la clase, luego el operador '::' y por último indicamos en nombre del método estático a llamar.
- Las propiedades estáticas no pueden ser accedidas a través del objeto utilizando el operador flecha (->).
- **Un método estático es lo más parecido a una función de un lenguaje estructurado. Solo que se lo encapsula dentro de una clase.**



Métodos y atributos estáticos

```
<?php
class Pagina {

    //Atributos
    public $nombre = "UNEWEB";
    public static $url =

"www.uneweb.edu.ve";

    //Metodos
    public function bienvenida ()
    {
        echo "Bienvenidos a
<b>". $this->nombre . "</b> la pagina es <b>". self::$url. "<b>";
    }
}
```

Métodos y atributos estáticos

```
public static function bienvenida2 ()
{
    echo "Bienvenidos ".

self::$url;
}

class Web extends Pagina{
}

/* Llamado 1 */
$pagina = new Pagina();
$pagina->bienvenida();
/* */
/* Llamado 2 (metodo estatico) sin llamar la clase y esta heradado */
//Web::bienvenida2();
?>
```

Interfaces de Objetos

¿Que son las interfaces?



Las interfaces de objetos permiten crear código con el cual especificamos qué métodos deben ser implementados por una clase.

Las interfaces son definidas utilizando la palabra clave *interface*, de la misma forma que con clases estándar, pero sin métodos que tengan su contenido definido.

Todos los métodos declarados en una interfaz deben ser públicos.

Interfaces de Objetos

Hablemos de los implements



Para implementar una interfaz, se utiliza el operador **implements**. Todos los métodos en una interfaz deben ser implementados dentro de la clase; el no cumplir con esta regla resultará en un error fatal. Las clases pueden implementar más de una interfaz si se deseara, separándolas cada una por una coma.

Las interfaces se pueden extender al igual que las clases utilizando el operador **extends**. La clase que implemente una interfaz debe utilizar exactamente las mismas estructuras de métodos que fueron definidos en la interfaz. De no cumplir con esta regla, se generará un error fatal.

Interfaces de Objetos

Ejercicio propuesto



Vamos a realizar una aplicación para un automóviles donde se deberá tener gasolina para poder encender el automóvil, además este podrá usarse pero gastara gasolina y una ves que su tanque este vacío se va a apagar.

Interfaces de Objetos

```
<?php
```

```
interface Auto
```

```
{
```

```
    public function encender();
```

```
    public function apagar();
```

```
}
```

```
interface gasolina extends Auto
```

```
{
```

```
    public function vaciarTanque();
```

```
    public function llenarTanque($cant);
```

```
}
```


Interface de Objetos

```
class Deportivo implements gasolina
{
    private $estado = false;
    private $tanque = 0;

    public function estado()
    {
        if($this->estado)
            print "El auto esta encendido y tiene " . $this->tanque . " de litros
en el tanque.";
        else
            print "El auto esta apagado<br>";
    }
}
```

Interfaces de Objetos

```
public function encender()  
{  
    if($this->estado)  
    {  
        print "No puedes encender el auto dos veces <br>";  
    }  
    else  
    {  
        if($this->tanque <= 0)  
        {  
            print "Usted no puede encender el auto porque el tanque esta vacio<br>";  
        }  
        else  
        {  
            print "Auto encendido <br>";  
            $this->estado = true;  
        }  
    }  
}
```

Interfaces de Objetos

```
public function apagar()  
{  
    if($this->estado)  
    {  
        print "Auto apagado <br>";  
        $this->estado = false;  
    }  
    else  
    {  
        print "El auto ya esta apagado <br>";  
    }  
}
```

```
public function vaciarTanque()  
{  
    $this->tanque = 0;  
}
```

Interfaces de Objetos

```
public function llenarTanque($cant)
{
    $this->tanque = $cant;
}
```

```
public function usar($km)
{
    if($this->estado)
    {
        $reducir = $km / 3;
        $this->tanque = $this->tanque - $reducir;
        if($this->tanque <=0)
            $this->estado = false;
    }
}
```

Interfaces de Objetos

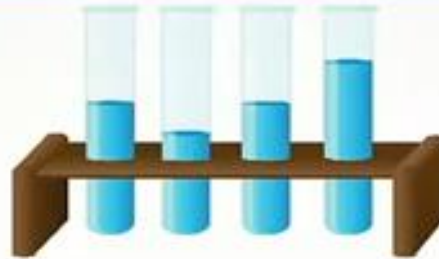
```
else
{
    print "El auto esta apagado y no se puede usar <br>";
}
}
```

```
$obj = new Deportivo();
$obj->llenarTanque(100);
$obj->encender();
$obj->usar(100);
$obj->estado();
```

```
?>
```

Clases y métodos abstractos

¿Que son las clases abstractas?



Una clase abstracta tiene por objetivo agrupar atributos y métodos que luego serán heredados por otras subclases.

Las clases definidas como abstractas no se pueden instanciar y cualquier clase que contiene al menos un método abstracto debe ser definida como tal. Los métodos definidos como abstractos simplemente declaran la firma del método, pero no pueden definir la implementación.

Clases y métodos abstractos

También hay métodos abstractos



Si queremos que las subclases implementen comportamientos obligatoriamente podemos definir métodos abstractos.

Un método abstracto se declara en una clase pero no se lo implementa.

Clases y métodos abstractos

```
<?php
```

```
    abstract class Molde{  
        abstract public function ingresarNombre($nombre);  
        abstract public function obtenerNombre();  
    }
```

```
    class Persona extends Molde{  
        private $mensaje = "Hola alumnos de UNWEB";  
        private $nombre;  
  
        public function mostrar(){  
            print $this->mensaje;  
        }  
    }
```


Clases y métodos abstractos

```
        public function ingresarNombre($nombre, $username = " Uw"){
            $this->nombre = $nombre . $username;
        }

        public function obtenerNombre(){
            print $this->nombre;
        }

    }

    $obj = new Persona();
    $obj->ingresarNombre("Carlos","Perez");
    $obj->obtenerNombre();
```

?>

Funcionamiento de autoload

¿Que es un autoload?



Muchos desarrolladores que escriben aplicaciones orientadas a objetos crean un **archivo** fuente PHP para cada definición de clase. Una de las mayores molestias es **tener que hacer una larga lista de includes al comienzo de cada script.**

Se puede definir una función que es automáticamente invocada en caso de que se esté intentando utilizar una clase/interfaz que todavía no haya sido definida. Al invocar a esta función el motor de scripting da una última oportunidad para cargar la clase antes que PHP falle con un error.

Funcionamiento de autoloading

¿Que vamos hacer?



Usaremos la función [spl_autoload_register\(\)](#), lo que va hacer es auto-cargar clases desde una función donde intentemos cargar los archivos donde se encuentran esas clases.

Funcionamiento de autoload

- Crear en su directorio de trabajo, una carpeta llamada “clases”.
- Crear dentro la carpeta clases dos archivos PHP, identificados con los nombres:
 - “Persona.php”
 - “Auto.php”

Funcionamiento de autoload

- Contenido de index.php

```
<?php
```

```
    function autoload($clase){  
        include "clases/" . $clase . ".php";  
    }
```

```
spl_autoload_register("autoload");
```

```
Persona::mostrar("Hola mundo");
```

```
echo "<br>";
```

```
Auto::mostrar("UNEWEB")
```

```
?>
```

Funcionamiento de autoload

- Contenido de \clases\persona.php

```
<?php
```

```
    class Persona{
```

```
        public function mostrar($mensaje){
```

```
            echo "La persona esta diciendo: "
```

```
        . $mensaje;
```

```
    }
```

```
}
```

```
?>
```

Funcionamiento de autoload

- Contenido de \clases\auto.php

```
<?php
    class Auto{
        public function mostrar($mensaje){
            echo "El auto esta diciendo: " .
$mensaje;
        }
    }
?>
```

Funcionamiento de autoload

- Otra forma de invocarlo es:

`localhost/micarpeta/index.php?action=clases`

y el `index.php` colocar:

Funcionamiento de autoload

```
<?php
```

```
    //$_GET['action'];
```

```
function autoload($clase){  
    include $_GET['action'] . "/" . $clase . ".php";  
}
```

```
spl_autoload_register("autoload");
```

```
Persona::mostrar("Hola mundo");
```

```
echo "<br>";
```

```
Auto::mostrar("UNEWEB")
```

```
?>
```

Uso de Traits

¿Que son los traits?



Personalmente los considero como fragmentos de códigos que pueden ser reusados en clases, los cuales pueden tener un poco de similitud con una clase.

Las clases pueden permitir usar tantos traits sean posibles simplemente separándolos con una `,` estaría funcionando de manera correcta.

Uso de Traits

Mas sobre traits



Un **Trait** es similar a una clase, pero con el único objetivo de agrupar funcionalidades muy específicas y de una manera coherente. No se puede instanciar directamente un Trait. Es por tanto un añadido a la herencia tradicional, y habilita la composición horizontal de comportamientos; es decir, permite combinar miembros de clases sin tener que usar herencia.

Uso de Traits

```
<?php
```

```
trait Persona2{  
    public $nombre;  
  
    public function mostrarNombre(){  
        echo $this->nombre;  
    }  
    abstract function asignarNombre($nombre);  
}
```

```
trait Trabajador{  
    protected function Hola(){  
        echo "es del trabajador";  
    }  
}
```

Uso de Traits

```
class Persona{  
    use Persona2, Trabajador;  
  
    public function asignarNombre($nombre){  
        $this->nombre = $nombre;  
    }  
  
    public function Hola(){  
        echo "es de la clase";  
    }  
}
```

Uso de Traits

```
$persona = new Persona();  
    $persona->asignarNombre("Jose ");  
    $persona->mostrarNombre();  
    echo "<br>";  
    $persona->Hola();
```

?>

Namespaces

¿Que son los nombres de espacio?



Los **namespaces** son sin duda alguna una de las innovaciones que ha “revivido” a la comunidad de PHP, estos hacen que la manera de programar con este lenguaje sea mas sencilla y a la vez mas cómoda.

Los nombres de espacio nos ayudan a tener que colocar “**sub-nombres**” a nuestros archivos, clases o funciones. Sin duda alguna es una gran ventaja para todos los que programan con la POO.

Namespaces

Ejemplo de los namespaces



Sin duda alguna unos de los ejemplos que podemos tomar es: ¿Cuántas veces hemos tenido que cambiar nombres de clases por ejemplo, “**Registro**” a “**Registro_estudiante**” porque ya exista otra clase con el mismo nombre y por lo tanto podíamos tener una colisión.

Namespaces

Crear una carpeta nueva llamada “Api”.

c:\AppServ\www\micarpeta\Api

Acceder a la carpeta api y crear dos nuevas carpetas identificadas como: “Models” y “Controllers”

Acceder a la carpeta “Models” y crear el archivo PHP: “Persona.php”.

Acceder a la carpeta “Controller” y crear el archivo PHP: “PersonasControllers.php”.

Namespaces

A continuación se indica contenido del archivo
“index.php”:

```
<?php
```

```
// SIN AUTOLOAD
```

```
    require_once "api/Models/Persona.php";
```

```
    require_once
```

```
"api/Controllers/PersonasControllers.php";
```

Namespaces

```
// CON AUTOLOAD
```

```
/*
```

```
    spl_autoload_register(function($clase){
```

```
        $ruta = "api/" . str_replace("\\", "/", $clase) . ".php";  
        print $ruta."<br>";
```

```
        if (is_readable($ruta)){  
            require_once $ruta;  
        }else{  
            echo "el archivo no existe";  
        }  
    }
```

```
});
```

```
*/
```

Namespaces

```
// SIN AUTOLOAD
```

```
Models\Persona::Hola();  
Controllers\Persona::Hola();
```

```
// CON AUTOLOAD
```

```
/*  
Models\Persona::Hola();  
Controllers\PersonasControllers::Hola();  
*/
```

```
?>
```

Namespaces

A continuación se indica contenido del archivo "Persona.php":

```
<?php namespace Models;  
    class Persona{  
        public static function Hola(){  
            echo "Hola soy la persona del  
modelo. <br>";  
        }  
    }  
?>
```

Namespaces

A continuación se indica contenido del archivo
“PersonasControllers.php”:

```
<?php namespace Controllers;  
// SIN AUTOLOAD  
        class Persona{  
// CON AUTOLOAD  
//      class PersonasControllers{  
                public static function Hola(){  
                        echo "Soy la persona del  
controlador.<br>";  
                }  
        }  
?>
```

Php POO y Bases de Datos

```
CREATE DATABASE UBICACION;
```

```
USE UBICACION;
```

```
CREATE TABLE ANIMAL(ID INT AUTO_INCREMENT,  
NOMBRE VARCHAR(50) NOT NULL,  
DESCRIPCION TEXT NOT NULL,  
PRIMARY KEY(ID));
```

```
DESCRIBE ANIMAL;
```

Field	Type	Null	Key	Default	Extra
ID	int(11)	NO	PRI	NULL	auto_increment
NOMBRE	varchar(50)	NO		NULL	
DESCRIPCION	text	NO		NULL	

```
INSERT INTO ANIMAL(NOMBRE, DESCRIPCION)  
VALUE  
( 'GATO','FELINO'),  
( 'LINCE','FELINO'),  
( 'TIGRE','FELINO'),  
( 'LEOPARDO','FELINO');
```

Php POO y Bases de Datos

```
<?php class conexion  
// archivo: conexion.php  
{
```

```
    public $server;
```

```
function __construct()  
{
```

```
    $this->server = "localhost";// NOMBRE SERVIDOR
```

```
    $this->usuario = "root";      // USUARIO MYSQL
```

```
    $this->password = "";          // PASSWORD DE  
USUARIO
```

```
    $this->base = "UBICACION"; // NOMBRE DE LA BASE DE  
DATOS
```


Php 4 POO y Bases de Datos

```
$this->link=mysqli_connect($this->server,  
$this->usuario,  
$this->password,  
$this->base);
```

```
}
```

```
public function ejecutar($sql)  
{  
    $x=mysqli_query($this->link,$sql);  
    return $x;
```

```
}
```

```
}
```

```
?>
```

Php POO y Bases de Datos

```
<?php include 'conexion.php';  
/* archivo: index.php  
   se instancia la clase */  
$conex = new conexion();  
$sql = "select * from animal";  
$x= $conex->ejecutar($sql);  
    while ($vec = mysqli_fetch_array($x)){  
        print "Nombre: " . $vec[0] . "-" . $vec[1] . "-" .  
$vec[2] . "<br>";  
    } ?>
```

Php POO, librería JPGRAPH

Descargar la librería JPGRAPH, desde Internet:

The screenshot shows the JpGraph website interface. The browser's address bar displays 'jpgraph.net'. The website header features the JpGraph logo with the tagline 'Most powerful PHP-driven charts' and a 'powered by Asial' badge. A navigation menu includes links for Home, News, Features, Download, Professional Version, Docs, Community Forum, and Contact. The 'Download' link is highlighted with a red circle. Below the navigation menu, there are three main sections: 'Chart Gallery' (with sub-sections for Line / Area, Bar, and Pie charts), 'Download JpGraph' (which is also circled in red), and 'Release Info'. To the right of these sections is an 'About' section describing JpGraph as an Object-Oriented Graph creating library for PHP5 and PHP7.0, and a 'Latest Pro-release' section dated 25 Jan, mentioning JpGraph-4.2.0-pro (4.2.0-pro) and supported PHP7.2.

jpgraph.net

Most powerful PHP-driven charts

powered by Asial

JpGraph

Home News Features **Download** Professional Version Docs Community Forum Contact

Download JpGraph

Release Info

Chart Gallery

Line / Area Bar Pie

About

JpGraph is an Object-Oriented Graph creating library for PHP5 (≥ 5.1) and PHP7.0 The library is completely written in PHP and ready to be used in any PHP scripts (both CGI/APXS/CLI versions of PHP are supported).

Latest Pro-release

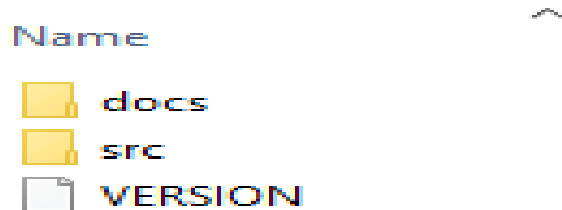
25 Jan: JpGraph-4.2.0-pro (4.2.0-pro)
Supported PHP7.2

Php POO, librería JPGRAPH

Una vez descargado, se procede a descomprimir y ubicar los archivos en el directorio JPGRAPH,



el cual a su vez contendrá los directorios:



Php POO, librería JPGRAPH

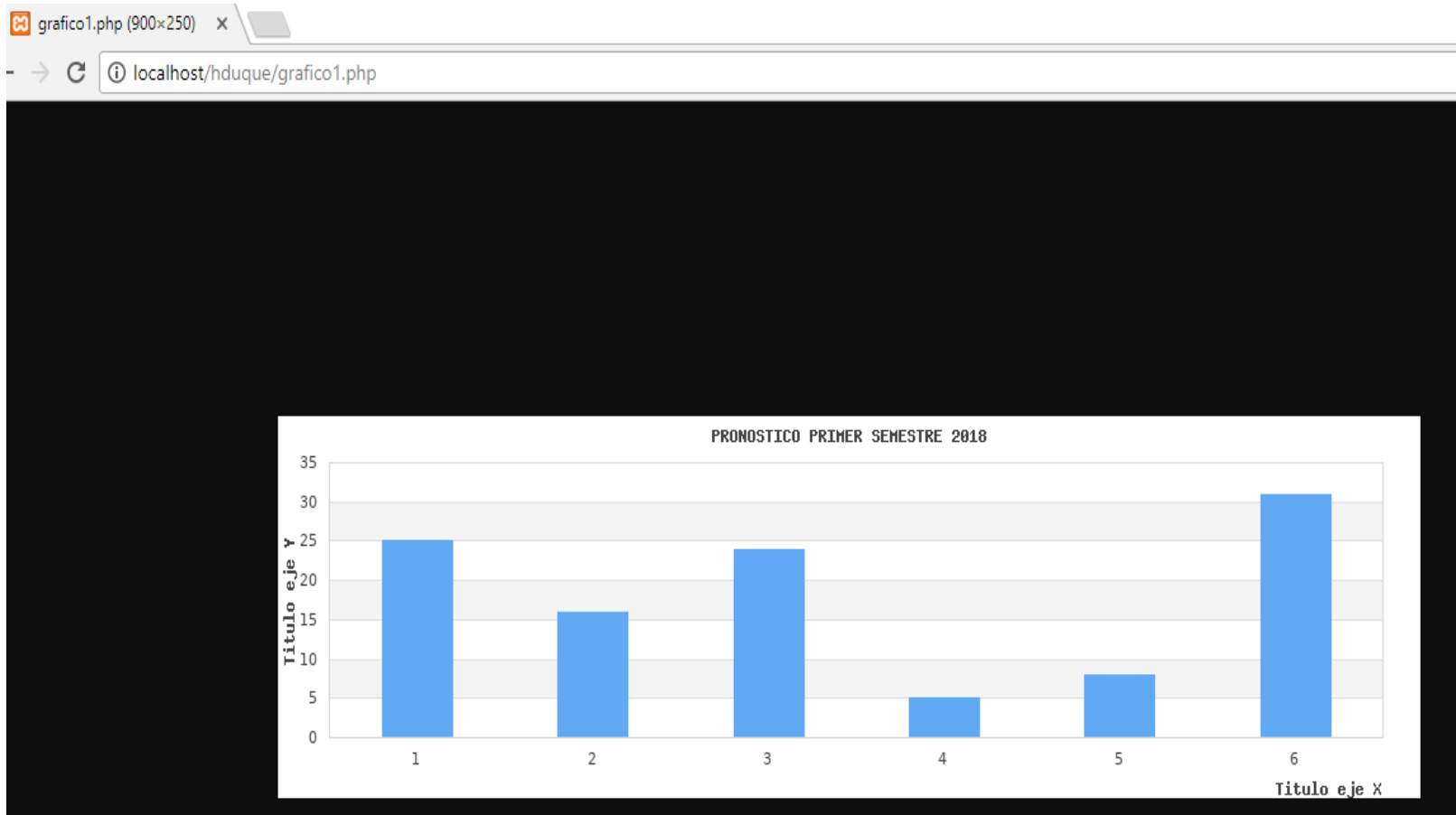
```
<?php
// ARCHIVO grafico1.php
date_default_timezone_set('America/Caracas');
require_once ('jpgraph/src/jpgraph.php');
require_once ('jpgraph/src/jpgraph_bar.php');
// Se define el array de datos
$datosy=array(25,16,24,5,8,31);
// Creamos el grafico tamaño ancho y alto
$grafico = new Graph(900,250);
$grafico->SetScale('textlin');
// Ajustamos los margenes del grafico----- (left,right,top,bottom)
$grafico->SetMargin(40,30,30,40);
// Creamos barras de datos a partir del array de datos
$bplot = new BarPlot($datosy);
```

Php POO, librería JGRAPH

```
// Configuramos color de las barras
$bplot->SetFillColor('#479CC9');
// Queremos mostrar el valor numerico de la barra
$bplot->value->Show();
//Añadimos barra de datos al grafico
$grafico->Add($bplot);
// Configuracion de los titulos
$grafico->title->Set('PRONOSTICO PRIMER SEMESTRE 2018');
$grafico->xaxis->title->Set('Titulo eje X');
$grafico->yaxis->title->Set('Titulo eje Y');
$grafico->title->SetFont(FF_FONT1,FS_BOLD);
$grafico->yaxis->title->SetFont(FF_FONT1,FS_BOLD);
$grafico->xaxis->title->SetFont(FF_FONT1,FS_BOLD);
// Se muestra el grafico
$grafico->Stroke();
?>
```

Php POO, librería JPGRAPH

- Muestra ejecución grafico1.php:



Php POO, librería JPGRAPH

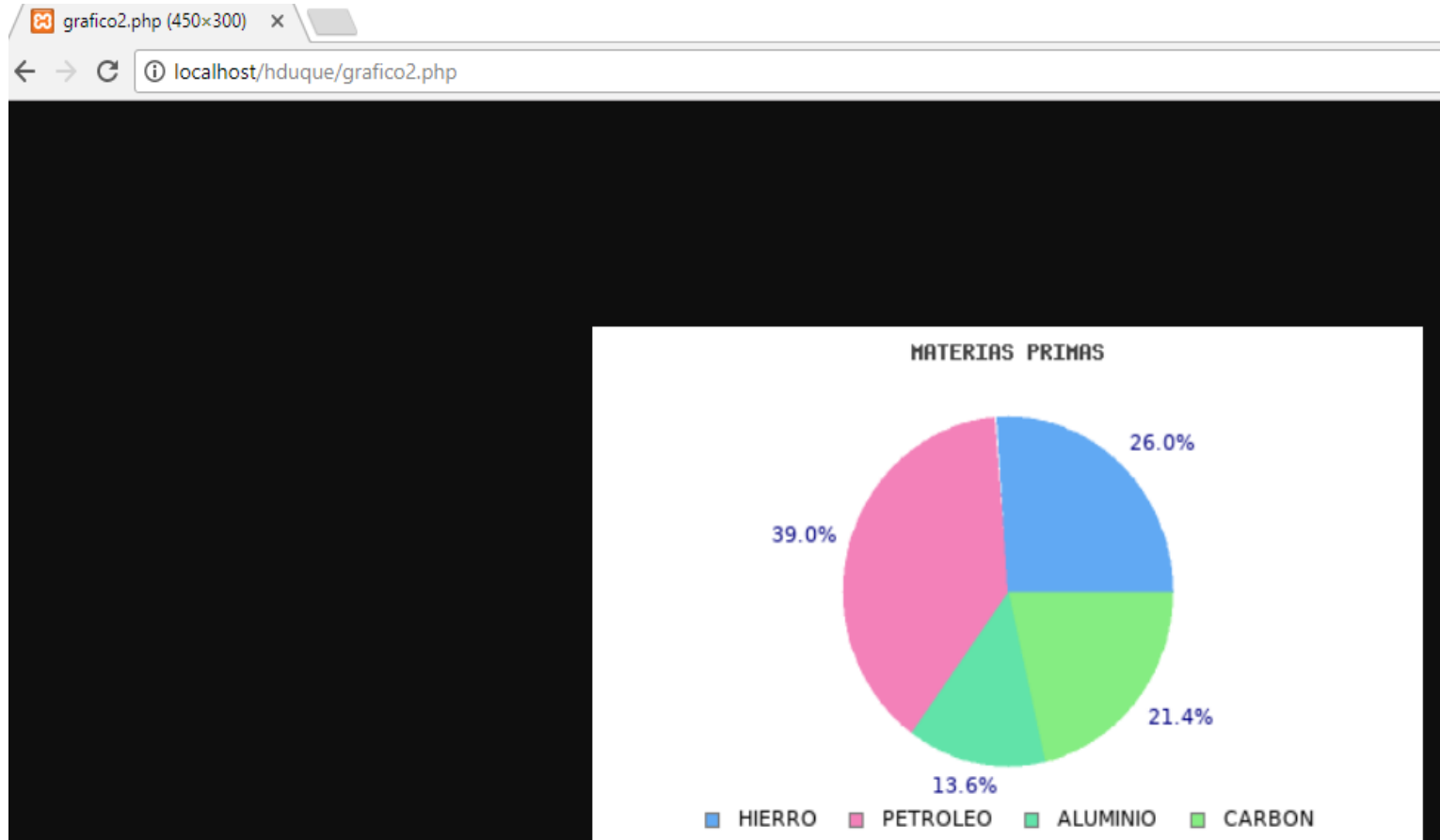
```
<?php
// ARCHIVO grafico2.php
date_default_timezone_set('America/Caracas');
require_once ("jpgraph/src/jpgraph.php");
require_once ("jpgraph/src/jpgraph_pie.php");
// Se define el array de valores y el array de la leyenda
$datos = array(40,60,21,33);
$leyenda =
array("HIERRO","PETROLEO","ALUMINIO","CARBON");
//Se define el grafico
$grafico = new PieGraph(450,300);
```


PHP POO, librería JGRAPH

```
//Definimos el titulo
$grafico->title->Set("MATERIAS PRIMAS");
$grafico->title->SetFont(FF_FONT1,FS_BOLD);
//Añadimos el titulo y la leyenda
$p1 = new PiePlot($datos); $p1-
>SetLegends($leyenda);
$p1->SetCenter(0.4);
//Se muestra el grafico
$grafico->Add($p1); $grafico->Stroke();
?>
```

PHP POO, librería JPGRAPH

- Muestra gráfico2.php:



Php POO, librería FPDF

Descargar la librería FPDF, desde internet:



The screenshot shows the homepage of the FPDF Library. At the top, there is a browser address bar displaying 'www.fpdf.org'. Below the address bar, on the left, is a vertical orange sidebar containing a logo of a blue elephant with 'FPDF' on its side, followed by flags for English, Spanish, French, and German. The sidebar lists navigation links: 'Principal', 'Descargas', 'Tutoriales', 'Manual', 'FAQ (PMF)', 'Scripts', 'Foro', and 'Enlaces'. The main content area on the right features the 'FPDF Library' title in a large blue font, with 'PDF generator' in red below it. A section titled '¿Qué es FPDF?' in purple text is followed by a paragraph explaining that FPDF is a PHP class for generating PDF documents directly from PHP. Below this, a list of features is provided under the heading 'FPDF tiene otras ventajas: funciones de alto nivel. Esta es una lista de sus principales características:'.

FPDF Library
PDF generator

¿Qué es FPDF?

FPDF es una clase escrita en PHP que permite generar documentos PDF directamente desde PHP, es decir, sin usar la biblioteca usted usarla para cualquier propósito y modificarla a su gusto para satisfacer sus necesidades.

FPDF tiene otras ventajas: funciones de alto nivel. Esta es una lista de sus principales características:

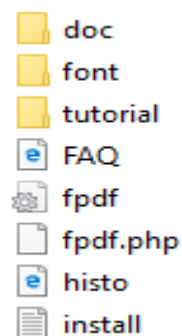
- Elección de la unidad de medida, formato de página y márgenes
- Gestión de cabeceras y pies de página
- Salto de página automático
- Salto de línea y justificación del texto automáticos
- Admisión de imágenes (JPEG, PNG y GIF)
- Colores
- Enlaces
- Admisión de fuentes TrueType, Type1 y codificación
- Compresión de página

Php POO, librería FPDF

Una vez descargado, se procede a descomprimir y ubicar los archivos en el directorio FPDF respectivo:



el cual a su vez contendrá los directorios:



Php POO, librería FPDF

```
<?php
```

```
// Archivo PDF1.PHP
```

```
require('fpdf181/fpdf.php');
```

```
$pdf=new FPDF();
```

```
$pdf->AddPage();
```

```
$pdf->SetFont('Arial','B',16);
```

```
$pdf->Cell(40,10,'MI PRIMERA PAGINA CON LA LIBRERIA  
FPDF');
```

```
$pdf->Output();
```

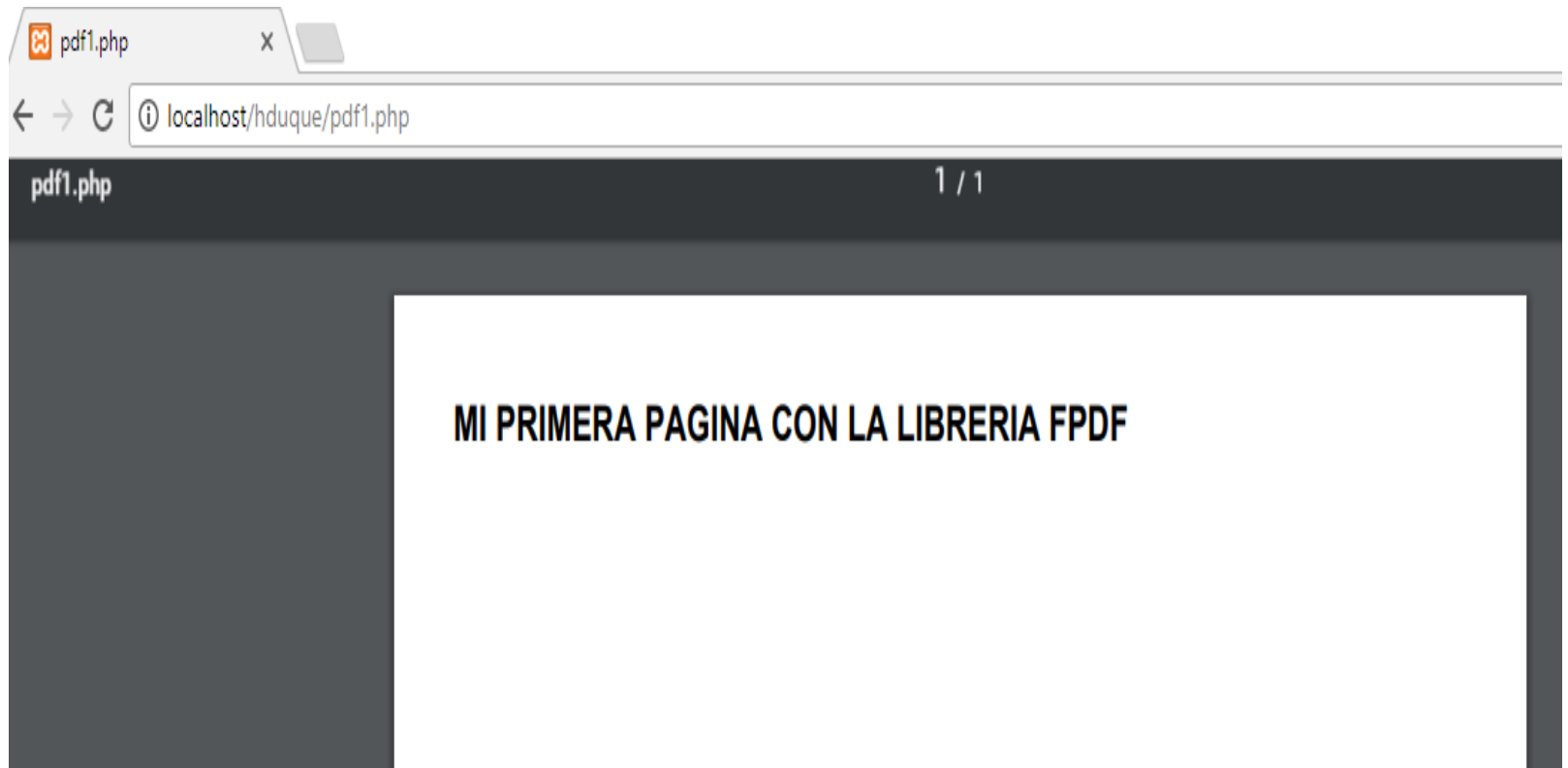
```
?>
```

Php POO, librería FPDF

- Lo primero que hacemos es incluir la librería `fpdf.php`
- En la línea `$pdf=new FPDF();` lo que estamos haciendo es crear el objeto FPDF. Si no ponemos nada entre los paréntesis, el objeto se creará con los valores por defecto, en este caso serían los siguientes: el tamaño de página es A4, el formato alargado y la unidad de medida el milímetro. Si queremos modificar estos parámetros sería en el siguiente orden `$pdf=new FPDF('formato','unidad de medida','tamaño');`
- En la línea `$pdf->AddPage();` añadimos una página.
- Con `SetFont();` le damos formato al texto diciendo el tipo de letra, si es en negrita o no, y el tamaño de la letra.
- Ya en la línea `$pdf->Cell();` empezamos a escribir el contenido de la página. Empezamos diciendo el ancho de la celda donde vamos a escribir, el alto de la celda, y el contenido de la celda.
- La última línea `$pdf->Output();` lo que hace es cerrar el archivo y enviarlo al navegador.

Php POO, librería FPDF

- Muestra resultado PDF1.PHP



Php POO, librería FPDF

```
<?php
// ARCHIVO PDF2.PHP

require('fpdf181/fpdf.php');
class PDF extends FPDF{
function Header()//Cabecera de página
{
    $this->Image('LOGO.jpg',10,8,33); //LOGO.JPG debe existir el archivo
    // en su defecto la línea de instrucción debe ser colocada en comentario.
    $this->SetFont('Arial','B',15); //Arial bold 15
    $this->Cell(80); //Movernos a la derecha
    $this->Cell(30,10,'TITULO',1,0,'C'); //Título
    $this->Ln(20); //Salto de línea
}
```


Php POO, librería FPDF

```
function Footer()  
//Pie de página  
{  
    $this->SetY(-15); //Posición: a 1,5 cm del final  
    $this->SetFont('Arial','I',8); //Arial italic 8  
    //Número de página  
    $this->Cell(0,10,'PAGINA '.$this->  
>PageNo().'{nb}',0,0,'C');  
}  
}  
$pdf=new PDF();
```

Php POO, librería FPDF

//Creación del objeto de la clase heredada

```
$pdf->AliasNbPages();
```

```
$pdf->AddPage();
```

```
$pdf->SetFont('Times','',12);
```

```
for($i=1;$i<=40;$i++)
```

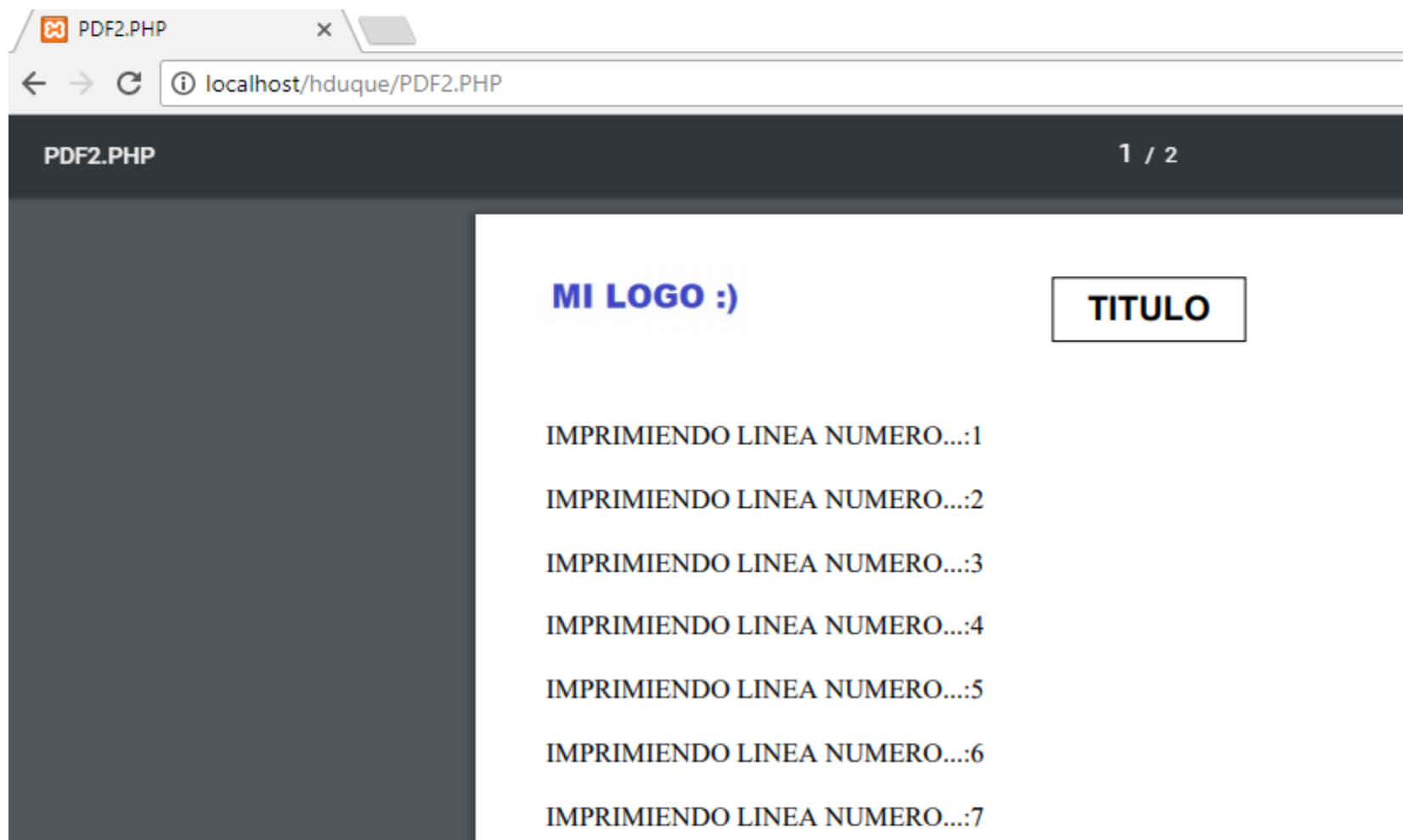
```
    $pdf->Cell(0,10,'IMPRIMIENDO LINEA  
NUMERO...:'. $i,0,1);
```

```
$pdf->Output();
```

```
?>
```

Php POO, librería FPDF

Muestra resultado PDF2.PHP



PHP POO, BD, Lista combinada

```
CREATE TABLE CIUDAD (ID_CIUADAD INT(3) AUTO_INCREMENT,  
CIUDAD VARCHAR(20) NOT NULL, PRIMARY KEY (ID_CIUADAD));
```

```
DESCRIBE CIUDAD;
```

Field	Type	Null	Key	Default	Extra
ID_CIUADAD	int(3)	NO	PRI	NULL	auto_increment
CIUDAD	varchar(20)	NO		NULL	

```
INSERT INTO CIUDAD (CIUDAD) VALUES  
("CARACAS"),  
("MARACAY");
```

PHP POO, BD, Lista combinada

```
CREATE TABLE PARROQUIA(ID_PARROQUIA INT(3) AUTO_INCREMENT,  
ID_CIUADAD INT(3) NOT NULL,  
PARROQUIA VARCHAR(20) NOT NULL, PRIMARY KEY(ID_PARROQUIA));
```

```
DESCRIBE PARROQUIA;
```

Field	Type	Null	Key	Default	Extra
ID_PARROQUIA	int(3)	NO	PRI	NULL	auto_increment
ID_CIUADAD	int(3)	NO		NULL	
PARROQUIA	varchar(20)	NO		NULL	

```
INSERT INTO PARROQUIA(ID_CIUADAD, PARROQUIA)  
VALUES  
(1,"CARICUAO"),  
(1,"23 DE ENERO"),  
(1,"LA VEGA"),  
(2,"LAS DELICIAS"),  
(2,"CENTRO");
```

PHP POO, BD, Lista combinada

```
CREATE TABLE PARROQUIA(ID_PARROQUIA INT(3) AUTO_INCREMENT,  
ID_CIUADAD INT(3) NOT NULL,  
PARROQUIA VARCHAR(20) NOT NULL, PRIMARY KEY(ID_PARROQUIA));
```

```
DESCRIBE PARROQUIA;
```

Field	Type	Null	Key	Default	Extra
ID_PARROQUIA	int(3)	NO	PRI	NULL	auto_increment
ID_CIUADAD	int(3)	NO		NULL	
PARROQUIA	varchar(20)	NO		NULL	

```
INSERT INTO PARROQUIA(ID_CIUADAD, PARROQUIA)  
VALUES  
(1,"CARICUAO"),  
(1,"23 DE ENERO"),  
(1,"LA VEGA"),  
(2,"LAS DELICIAS"),  
(2,"CENTRO");
```

PHP POO, BD, Lista combinada

SELECT * FROM CIUDAD;

ID_CIUADAD	CIUDAD
1	CARACAS
2	MARACAY

SELECT * FROM PARROQUIA;

ID_PARROQUIA	ID_CIUADAD	PARROQUIA
1	1	CARICUAO
2	1	23 DE ENERO
3	1	LA VEGA
4	2	LAS DELICIAS
5	2	CENTRO

PHP POO, BD, Lista combinada

Contenido programa : conexion1.php

```
<?php
```

```
// ARCHIVO: conexion1.php
```

```
class Conexion1{
```

```
var $hostname = "localhost"; //IP del servidor
```

```
var $database; //Nombre de la base de datos
```

```
var $username = "root";//usuario mysql
```

```
var $password = ""; //password de usuario mysql
```


PHP POO, BD, Lista combinada

```
function conectar($base){  
    print $base;  
    $this-> database=$base;  
  
    $link = mysqli_connect($this->hostname, $this->username, $this->password,$this->database);  
  
    if(!$link){  
        echo "conexion fallida";  
        exit;  
    }else{  
        mysqli_select_db($link,$this->database);  
    }  
    return $link;  
}  
?  
?>
```

PHP POO, BD, Lista combinada

Contenido programa: INDEX-14.php

```
<?php
```

```
// ARCHIVO: INDEX-14.PHP
```

```
// datos de conexion a la BD.
```

```
include_once("conexion1.php");
```

```
$conexion=new Conexion1();
```

```
$conn=$conexion->conectar("ubicacion");
```

```
// Obtener el $id_padre..
```

```
    $id_ciudad=$_POST['id_ciudad'];
```

```
    print $id_ciudad;
```

```
// Inicio Formulario .. PHP_SELF enviamos a si mismo (a este script).
```

```
    echo "<form action=\"".$_SERVER['PHP_SELF']."\"  
method=\"POST\">\n\n";
```

PHP POO, BD, Lista combinada

```
// Formar Select "Padre".
```

```
echo "<select name=\"id_ciudad\" onChange=\"this.form.submit()\">\n";
```

```
echo "<option value=\"\"> Seleccione un Item </option>\n";
```

```
$SQLconsulta_ciudad="SELECT * FROM ciudad";
```

```
$consulta_padre = mysqli_query($SQLconsulta_ciudad,$conn) or die(mysqli_error());
```

```
while ($registro_ciudad=mysqli_fetch_assoc($consulta_padre)){
```

```
    /* Se mira si el ID del registro es el mismo que el $id_ciudad que recibimos si  
    hemos cambiado el select parroquia.
```

```
    Se selecciona en consecuencia (selected) la opción elegida. */
```

```
    if ($id_ciudad == $registro_ciudad['id_ciudad']){
```

```
        echo "<option value=\"\".$registro_ciudad['id_ciudad'].\"\"\n"  
selected>\".$registro_ciudad['ciudad'].\"</option>\n";
```

```
    } else {
```

```
        echo "<option  
value=\"\".$registro_ciudad['id_ciudad'].\"\">\".$registro_ciudad['ciudad'].\"</option>\n"  
;
```

```
    }
```

```
}
```

PHP POO, BD, Lista combinada

```
echo "</select>\n\n";
mysqli_free_result($consulta_padre); // Liberar memoria usada por consulta.
// Formar Select "Parroquia"
echo "<select name=\"id_parroquia\">\n";
// Si $id_ciudad no tiene valor (caso de que no se ha seleccionado ninguna opción del
select hijo
// se muestra el mensaje de "seleccione un item" (del select padre).
if (!empty($id_ciudad)){
$SQLconsulta_parroquia="SELECT * FROM parroquia WHERE id_ciudad='$id_ciudad'";
$consulta_parroquia = mysqli_query($SQLconsulta_parroquia,$conn) or
die(mysqli_error());
// se mira el total de registros de la consulta .. si es 0 se muestra mensaje en el
select ..
if (mysqli_num_rows($consulta_parroquia) != 0){
While ($registro_parroquia=mysqli_fetch_assoc($consulta_parroquia)){
echo "<option
value=\"\".$registro_parroquia['id_parroquia'].\"\">\".$registro_parroquia['parroquia'].\"
</option>\n";
```

PHP POO, BD, Lista combinada

```
}  
    } else {  
        echo "<option value=\"\"> No hay registros  
para este Item </option>";  
    }  
    } else {  
        echo "<option value=\"\"> <-- Seleccione un  
Item </option>";  
    }mysqli_free_result($consulta_parroquia); //  
Liberar memoria usada por consulta.  
?>
```

GRACIAS